JK Assistance

An admission assistant for students in Azad Jammu & Kashmir

Manual · How it works · Building an agent from scratch · Interview questions and answers

A beginner-friendly guide

Author: Faisal Ramzan · Repository: github.com/faisalramzan3725/ajk-admission-assistant (private) · Version: September 2026

Contents

1. What JK Assistance is . . . 4

Who it helps . . . 4

What it covers . . . 4

2. How it works . . . 5

The building blocks . . . 5

How eligibility is checked . . . 6

The merit aggregate and the chance label . . . 6

How the chat understands a question . . . 7

Keeping the information fresh: the update agent . . . 7

Dealing with a broken official website . . . 7

3. User manual . . . 8

Install and run . . . 8

The portal, screen by screen . . . 8

Navigation . . . 8

Example questions to try . . . 9

For developers: API, MCP and tests . . . 9

Updating the information . . . 9

4. Building an agent from scratch (beginner tutorial) . . . 10

First, the key ideas in plain words . . . 10

Step 1 - Define the goal and the user . . . 10

Step 2 - Collect the official sources . . . 10

Step 3 - Turn facts into structured data . . . 11

Step 4 - Write the rule engine . . . 11

Step 5 - Add search over your documents (retrieval) . . . 11

Step 6 - Understand the question (router) and decide the action . . . 12

Step 7 - Let the LLM write the answer (grounded) . . . 12

Step 8 - Build the interfaces . . . 12

Step 9 - Test everything . . . 13

Step 10 - Make it an agent that keeps itself up to date . . . 13

Step 11 - Guardrails and lessons learned . . . 13

5. Interview preparation . . . 14

Your 60-second pitch . . . 14

A. Questions about the project . . . 14

B. Questions about AI, LLMs and RAG . . . 15
C. Questions about agents . . . 15
D. Questions about engineering and testing . . . 16
E. Questions about ethics and limits . . . 16
F. General and behavioural questions . . . 16
Tips for answering well . . . 17
6. Glossary . . . 18

1. What JK Assistance is

Every year thousands of students in Azad Jammu & Kashmir (AJK) finish their intermediate (12 years of education) and face the same questions: *Which programs can I apply for with my group and marks? What MDCAT or ECAT score do I need? Which documents do I prepare? Where do I apply, and by when?* The official answers are spread across many university websites, scanned PDFs and notices - and one key admission site (the Joint Admission Committee portal for MBBS/BDS) does not even open.

JK Assistance puts all of this in one place. A student enters their group and marks; the assistant shows every program they can apply to, checks each requirement against their own marks, calculates the MBBS/BDS merit aggregate, lists the documents and fees, explains how to apply step by step, and shows the admission dates with links.

Who it helps

- Students who have just finished FSc (pre-medical or pre-engineering), ICS, FA, I.Com, DAE or A-level and have not decided where to go.
- Parents and teachers who guide them.
- Admission offices, as a single place where students can find correct, sourced information.

What it covers

Area	Details
Institutions	University of AJK (UAJK, Muzaffarabad), Mirpur University of Science & Technology (MUST), the AJK public medical colleges (AJKMC, MBBSMC, Poonch Medical College) and SZABMU federal medical colleges with AJK-quota seats.
Programs	40 program groups: engineering, computing, medicine, dentistry, health sciences, natural and social sciences, business, law, education, arts.
Entry tests	MDCAT (out of 180; MBBS needs 99, BDS 90), ECAT (out of 400; PEC engineering needs 132), MUST SEET (33%), UAJK entry test (50%), HEC LAT (50%).
Dates	Last dates, merit lists and MDCAT schedule, each with a clickable link to where the student acts.
Languages	Questions in English or Roman Urdu.

A quick example

A student types: "*I did FSc pre-medical 946/1100, matric 1010/1100, MDCAT 140/180 - I want MBBS or DPT.*"
JK Assistance answers: **MBBS at the AJK public colleges - Eligible.** HSSC needs ≥ 60%, you have 86% (pass). MDCAT needs ≥ 55% = 99 / 180, you have 140 / 180 = 77.78% (pass). Merit aggregate = 0.5 × 77.78 + 0.4 × 86 + 0.1 × 91.82 = **82.472%**. Then it lists DPT options, the documents, the steps to apply and the upcoming dates.

2. How it works

The most important idea: **the rules decide, the AI only explains.** Eligibility, marks, fees and dates are computed by ordinary code from carefully checked data. A large language model (Claude) is used only to phrase the answer nicely - and only when an API key is configured. This makes every answer correct, repeatable and traceable to an official source.

From official websites to a student's answer

1. Official sources
University websites, PM&DC notices, prospectuses, scanned fee sheets (saved in raw/ and documents/)

↓

2. Knowledge files (YAML)
Checked facts: programs, minimum marks, tests (max/min), fees, steps, dates, each with its source

↓

3. Eligibility engine
Compares the student's marks with every program's rules: pass / fail / pending / check

↓

4. Chat router + decision
Understands the question; decides GUIDE, ASK_INFO, ALERT, ESCALATE or OFF_TOPIC

↓

5. Evidence search (BM25)
Finds the official text behind the answer (1,450 indexed chunks)

↓

6. Answer
Claude writes from the result + evidence, or the built-in renderer formats it

↓

7. Interfaces
Web portal, REST API, MCP tools for Claude Desktop / Claude Code

The building blocks

Block	File	What it does (in simple words)
Knowledge	knowledge/*.yaml	The 'rule book': every program's accepted groups and minimum marks, every test's maximum and passing marks, fees, steps and dates - each with its source link.
Knowledge loader	jk_assistance/knowledge.py	Reads the rule book and builds a search index over all official pages and documents.
Eligibility engine	jk_assistance/advisor.py	Converts marks to percentages, checks each requirement, gives a verdict, computes the merit aggregate and a chance estimate.
Chat router	jk_assistance/assistant.py	Reads a free-text question, pulls out group and marks, and picks the right engine.

Block	File	What it does (in simple words)
Decision layer	jk_assistance/decision.py	Stops unsafe or unanswerable cases: sensitive topics and exceptions go to the admission office.
LLM composer	jk_assistance/llm.py	Optional: asks Claude to write the answer using only the engine result and numbered evidence.
API + portal	jk_assistance/api.py, web/index.html	The website students use, and endpoints for other programs.
MCP server	mcp_server/server.py	Lets AI assistants (Claude Desktop, Claude Code) use JK Assistance as tools.
Update agent	scripts/update_agent.py	Checks the 23 official sources daily and reports changes for a person to verify.

How eligibility is checked

For each program the engine builds a small checklist. Each row has a **requirement**, what is **needed**, what the student **has**, and a **status**:

Requirement	Needed	You	Status
Intermediate group	F.Sc. Pre-Medical	F.Sc. Pre-Medical	pass
HSSC marks	≥ 60%	86%	pass
MDCAT	≥ 55% = 99 / 180	140 / 180 = 77.78%	pass
AJK State Subject	required for AJK-national seats	yes	pass

Then it gives one of four verdicts:

- **Eligible** - every requirement passes.
- **Eligible if you pass the test** - only an entry test is still missing (the student has not taken it yet). The engine never rejects a student just because a test is not taken.
- **Check with the office** - a rule is not published (for example BS Nursing minimum marks).
- **Not eligible** - a requirement fails, and the reason is shown (for example "HSSC 58% vs ≥ 60%").

Some programs accept **any one** of several tests. MUST engineering accepts its own SEET *or* another PEC-approved test such as ECAT. If a student's ECAT is below 132/400 but they have not taken SEET yet, they are still "eligible if you pass the test".

The merit aggregate and the chance label

For MBBS/BDS the official PM&DC formula is used: **MDCAT 50% + HSSC 40% + SSC 10%**. For other programs no university publishes closing merits, so the **chance** label is a clearly marked *heuristic*: it looks at how far the student is above the minimum and how competitive the program is. For ECAT programs it compares the score with UET's official average for the student's group (for example 203/400 for pre-engineering in Fall 2026).

How the chat understands a question

Student asks	The assistant decides	Result
"FSc pre-engineering 880/1100, ECAT 210 - engineering"	Marks are given -> admission advice	Programs with checklists
"MDCAT passing marks"	A test is named, no marks -> test information	Max 180, MBBS 99, BDS 90, format, fee
"How to apply for MBBS in AJK"	Steps question + medicine -> how to apply	Steps, documents, working contacts
"UAJK last date"	Date question -> deadlines	Dates with links (ALERT if close)
"I missed the deadline, can I still apply?"	Exception -> ESCALATE	Contact the admission office
"How do I get my transcript?"	Not about admission -> OFF_TOPIC	Polite redirect

Keeping the information fresh: the update agent

Admission dates and fees change every year. The **update agent** runs every day (through GitHub Actions) and, for each of the 23 official sources, it: fetches the page, compares it with yesterday's copy, looks for new dates, fees, rule sentences and new PDF notices, checks that every link shown to students still works, and flags when the calendar needs the next admission cycle. When something changed it opens a **pull request** with a report. A person checks the official source and approves the change. The agent never edits the rules by itself, because fees, dates and minimum marks are official facts.

> **Why a human approves**
>
> An AI or a script can misread a notice. By letting the agent *find and highlight* changes but letting a person *confirm* them, every answer a student sees stays traceable to a verified official source.

Dealing with a broken official website

The JAC admission portal (jac.ajk.gov.pk) returns an error. Instead of sending students to a dead link, JK Assistance marks it as not working and points to working official channels: PM&DC (the regulator), SZABMU Islamabad (which conducts MDCAT for AJK and runs federal colleges with AJK-quota seats) and the admission cells of Poonch Medical College and MBBS Medical College Mirpur. The update agent reports when the JAC site comes back.

3. User manual

Install and run

1. Install Python 3.11 or newer.
2. Open a terminal in the project folder and install the libraries: `pip install -r requirements.txt`
3. Start the server: `python -m uvicorn jk_assistance.api:app --port 8010`
4. Open **http://localhost:8010** in a browser. API documentation is at **http://localhost:8010/docs**.

> **If the port is busy**
> The error *[Errno 10048] ... 8010* means another program already uses port 8010. Stop it, or start on another port with `--port 8011`.

Optional: copy `.env.example` to `.env` and set `ANTHROPIC_API_KEY` to let Claude write the chat answers. Without it everything still works; answers are formatted by the built-in renderer.

The portal, screen by screen

Screen	What the student does
1. Find my programs	Choose the intermediate group; enter HSSC and SSC marks as *obtained / out of*; enter test scores if taken (MDCAT out of 180, ECAT out of 400, UAJK test, SEET, LAT); optionally choose an interest, a university, age, State Subject and Mathematics. Press **Check my eligibility**.
Results	A summary (how many programs are open, the next deadline), then one card per program with the needed-vs-yours table, the verdict, chance, fees and seats, and the steps to apply. Buttons: **Edit my marks**, **Start over**, **Next: how to apply**.
2. How to apply	Steps, a tickable document checklist, dates, age limit, reserved seats, scholarships, fee rules and contacts for UAJK, MUST and MBBS/BDS.
Entry tests	One card per test: total marks, minimum marks, format, fee and dates.
Admission dates	Upcoming dates first. Each title is a link, with a **Go** button (portal, merit lists) and a link to the official notice.
Ask a question	Chat in English or Roman Urdu, with the sources used.

Navigation

- ← **Back** returns to the previous screen (the browser's back button and Alt+← also work, and results are kept).
- ⌂ **Home** goes to "Find my programs".
- ✕ **Clear** empties the current screen: marks and results, the conversation, or the checklist.
- Each screen has its own address, e.g. `/#dates`, which can be shared or bookmarked.

Example questions to try

- I did FSc pre-medical 946/1100, matric 1010/1100, MDCAT 140/180 - I want MBBS or DPT
- FSc pre-engineering 880/1100, ECAT 210 - engineering
- ICS 64% computer science
- MDCAT passing marks · ECAT total marks
- How do I apply to UAJK? · mujhe MUST mein admission kaise milega
- UAJK last date

For developers: API, MCP and tests

What	How
Eligibility API	POST /api/advise with group, hssc, ssc, tests, interests ... (see the /docs page)
Other endpoints	/api/tests, /api/how-to-apply/{uajk|must|jac}, /api/deadlines, /api/programs, /api/search, /api/chat, /api/health
MCP server	python mcp_server/server.py (tools: check_admission, get_test, how_to_apply, admission_dates, list_programs, search_admission_sources, ask_jk_assistance)
Tests	python -m pytest -q (49 tests)
Sample notebook	notebooks/JK_Assistance_Samples.ipynb - rebuild with python scripts/build_notebook.py
Update agent	python scripts/update_agent.py --no-llm (report in reports/update_report.md)

Updating the information

1. Read the update agent's report (or the pull request it opened).
2. Open the official source link and confirm the change.
3. Edit `knowledge/deadlines.yaml` (dates) or `knowledge/admissions.yaml` (programs, tests, fees, steps). Always include the source link.
4. Run `python -m pytest -q`, check the portal, and merge.

4. Building an agent from scratch (beginner tutorial)

This part explains, step by step, how to build an assistant like JK Assistance even if you are new to AI. The same recipe works for other domains: scholarship finders, visa guides, hospital help desks, HR policy assistants.

First, the key ideas in plain words

Term	Meaning
LLM (large language model)	A model such as Claude that reads and writes text. Very good at language, but it can invent facts if not grounded.
Grounding	Giving the model the exact facts to use, so it does not guess.
RAG (retrieval-augmented generation)	Search your own documents first, then give the found text to the model to answer from.
Rule engine	Ordinary code with if/else rules. Perfect for facts like "HSSC ≥ 60%".
Agent	A program that observes, decides and acts towards a goal, often in a loop, using tools. JK Assistance's update agent watches websites, decides what changed and acts by reporting it.
Tool / MCP	A function an AI can call. MCP (Model Context Protocol) is a standard way to offer tools to AI apps.
Human in the loop	A person approves important actions (here: changes to official dates and fees).

Step 1 - Define the goal and the user

Write one sentence: "*Help a student who has finished 12th grade in AJK decide where to apply, check eligibility and follow the steps.*" List the questions users really ask. Decide what the assistant must **not** do (here: invent cut-off marks, answer non-admission topics, send people to dead links).

Step 2 - Collect the official sources

Download every official page and document, and record where each came from. Keep the raw copies; they are your evidence.

```
import requests, bs4
def fetch(url):
    r = requests.get(url, headers={"User-Agent": "Mozilla/5.0"}, timeout=30)
    soup = bs4.BeautifulSoup(r.text, "lxml")
    for t in soup(["script", "style", "nav", "footer"]):
        t.decompose()                       # remove menus and scripts
    return soup.get_text("\n", strip=True)  # clean page text
```

Step 3 - Turn facts into structured data

Do not leave important facts inside long text. Put them in a clear format (YAML or JSON) that code can use, with the source next to each fact.

```
tests:
  mdcat:
    max_marks: 180
    passing:
      mbbs: {percent: 55, marks: 99}
      bds:  {percent: 50, marks: 90}
    merit_formula: {mdcat: 50, hssc: 40, ssc: 10}
    sources: ["https://pmdc.pk/Documents/law/Admissions Regulations-2025.pdf"]
programs:
  - {id: jac.mbbs, groups: [pre_med, a_level], min_percent: 60,
     test_rules: [{test: mdcat, min_percent: 55}]}
```

Step 4 - Write the rule engine

Use plain code for decisions that must be exactly right. Show the student both what is needed and what they have.

```
def check_program(prog, student):
    rows = []
    rows.append(("HSSC", f">= {prog['min_percent']}%", student["hssc"],
                 student["hssc"] >= prog["min_percent"]))
    for rule in prog["test_rules"]:
        score = student["tests"].get(rule["test"])          # None = not taken yet
        ok = None if score is None else score >= rule["min_percent"]
        rows.append((rule["test"], f">= {rule['min_percent']}%", score, ok))
    if any(ok is False for *_, ok in rows):  return "not eligible", rows
    if any(ok is None for *_, ok in rows):   return "eligible if you pass the test", rows
    return "eligible", rows
```

Step 5 - Add search over your documents (retrieval)

Split documents into chunks and index them. BM25 is a simple, fast keyword ranking method and a good first choice; vector embeddings can be added later.

```
from rank_bm25 import BM25Okapi
chunks = [...]                                      # pieces of official text
index = BM25Okapi([c.lower().split() for c in chunks])
scores = index.get_scores("mdcat passing marks".split())
top = sorted(zip(scores, chunks), reverse=True)[:6]  # evidence for the answer
```

Step 6 - Understand the question (router) and decide the action

Pull the useful details from the question (group, marks, test names), then route it: advice, test info, how to apply, dates - or stop early when a person must decide (exceptions, sensitive topics) or when the question is out of scope.

```
import re
def route(q):
    if re.search(r"harass|missed the deadline|extension|appeal", q, re.I): return "ESCALATE"
    if re.search(r"transcript|cgpa|degree issue", q, re.I):             return "OFF_TOPIC"
    if re.search(r"\d+\s*/\s*\d+|\d+\s*%", q):                          return "ADVICE"
    if re.search(r"mdcat|ecat", q, re.I):                               return "TEST_INFO"
    if re.search(r"last date|deadline", q, re.I):                       return "DATES"
    return "HOW_TO_APPLY"
```

Step 7 - Let the LLM write the answer (grounded)

Give Claude the engine's result and the evidence, and tell it clearly not to invent anything. If the call fails, fall back to your own formatted answer so the system always works.

```
import anthropic, json
SYSTEM = ("You are an admission assistant. Use ONLY the RESULT and EVIDENCE. "
          "Never invent marks, fees or dates. Cite evidence as [1], [2].")
def compose(question, result, evidence):
    client = anthropic.Anthropic()                  # reads ANTHROPIC_API_KEY
    resp = client.messages.create(
        model="claude-opus-5-5", max_tokens=16000, system=SYSTEM,
        messages=[{"role": "user", "content":
                   f"QUESTION: {question}\nRESULT: {json.dumps(result)}\nEVIDENCE: {evidence}"}])
    return "".join(b.text for b in resp.content if b.type == "text")
```

Step 8 - Build the interfaces

- **Web portal** for students (a form with labelled inputs, results cards, dates with links).
- **REST API** (FastAPI) so other apps can use the engine.
- **MCP server** so AI assistants can call your functions as tools.

```
from fastapi import FastAPI
app = FastAPI()
@app.post("/api/advise")
def advise(profile: dict):
    return check_all_programs(profile)          # the rule engine from Step 4
```

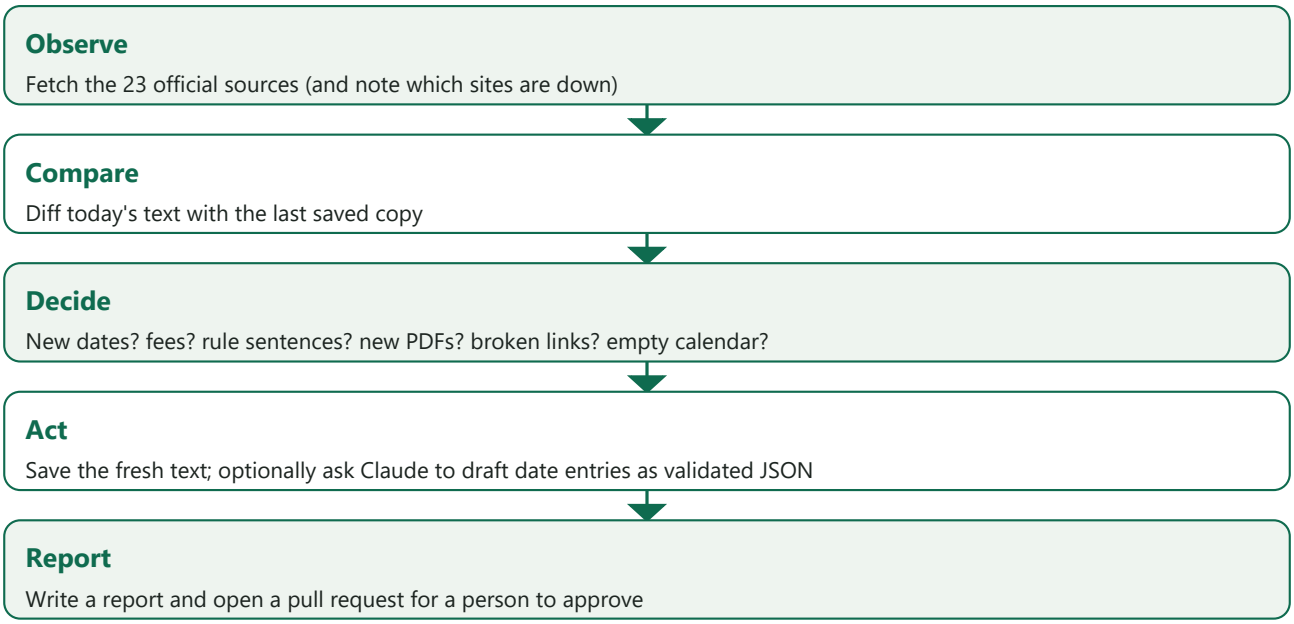
Step 9 - Test everything

Write a test for every rule and edge case: below the minimum, test not taken, the "any one test" rule, wrong group, age limit, and for the chat, that each sample question goes to the right place. Tests protect you when data or code changes.

```
def test_mdcat_below_99_fails_mbbs():
    r = advise({"group": "pre_med", "hssc": 80, "tests": {"mdcat": 95}})
    assert status(r, "jac.mbbs") == "not_eligible"
```

Step 10 - Make it an agent that keeps itself up to date

The update agent follows a classic agent loop: **observe** → **compare** → **decide** → **act** → **report**, then waits for the next run.



Step 11 - Guardrails and lessons learned

- Never let the model invent official numbers - compute them.
- Record the source and a trust level for every fact; label third-party information "verify".
- Handle broken websites honestly and give working alternatives.
- Ask for missing information instead of guessing (ASK_INFO).
- Send exceptions and sensitive cases to people (ESCALATE).
- Keep a human approval step for changes to official facts.
- Watch out for small data traps: "MUST" is also an English word; "pre-medical" is a group, not an interest; MDCAT 95 means 95/180, not 95%.

5. Interview preparation

This part helps you present JK Assistance in a job or scholarship interview. Answers are written in simple language; adapt them to your own words.

Your 60-second pitch

> **Say this when asked "Tell me about your project"**
>
> "Students in Azad Kashmir who finish intermediate often don't know where they can apply or what marks they need, because the information is spread across many websites and scanned PDFs, and one official admission portal doesn't even work. I built JK Assistance, an admission assistant that turns official sources into a checked rule book. A student enters their group and marks, and it shows every program they can apply to, with each requirement as needed versus their own marks - for example MDCAT 99 out of 180 versus their 140 - plus the MBBS merit aggregate, documents, fees, steps and dates with links. The key design choice is that rules decide and the AI only explains, so answers are exact and traceable. A daily agent watches 23 official sources and opens a pull request when something changes, and a person approves it. It has a web portal, an API, MCP tools and 49 automated tests."

A. Questions about the project

Q1. What problem does your project solve?

Students after 12th grade in AJK lack one reliable place to learn which programs they can join, what test scores they need and how to apply. JK Assistance gives a personalised answer from official sources in seconds, including marks comparisons, documents, fees and dates.

Q2. Who are the users, and how did you decide what to build?

Students who haven't decided where to go, plus parents and teachers. I listed the questions they actually ask - eligibility, test marks, documents, deadlines - and built one feature for each. I also narrowed the scope to admissions only so the tool stays focused and reliable.

Q3. Walk me through what happens when a student checks eligibility.

Their marks are converted to percentages (for example 946/1100 = 86%). For each of the 40 programs the engine checks the group, the minimum HSSC, Mathematics if required, the entry test, age and State Subject. Each check becomes a row of needed versus theirs with pass, fail, pending or check. The rows give a verdict, the MBBS aggregate is computed with the PM&DC formula, and the results are ranked by the student's interests.

Q4. Where does the data come from, and how do you know it's correct?

From official websites and documents: UAJK, MUST, PM&DC notices and regulations, SZABMU, the medical colleges. I saved the raw copies, extracted the text, read scanned PDFs manually, and stored each fact with its source link and a trust level. Third-party facts are marked 'verify'. Tests check that the data files are consistent.

Q5. What was the hardest part?

Messy, unreliable sources. Some fee sheets were scanned images, some pages only render with JavaScript, the JAC portal is blocked, and even a similarly named website (pmc.edu.pk) belonged to a different college. I handled each case explicitly: manual transcription, a real browser capture, working alternative channels, and verification of every domain.

Q6. What would you improve next?

Add the next admission cycle's dates as they are announced, transcribe the remaining scanned fee sheets, add Urdu-script answers, and, if universities publish closing merits, replace the heuristic chance with a real comparison.

B. Questions about AI, LLMs and RAG

Q7. Why didn't you let the AI answer everything directly?

LLMs can produce confident but wrong numbers. Admission decisions depend on exact values like 99 out of 180, so the numbers are computed by code from verified data. The model only turns the result into friendly language, and it must cite the evidence.

Q8. What is RAG and how did you use it?

Retrieval-augmented generation means searching your own documents first and giving the found text to the model. I split the official pages and documents into about 1,450 chunks and used BM25 keyword search to find the six most relevant pieces for each question.

Q9. Why BM25 and not vector embeddings?

The questions contain precise words like MDCAT, ECAT, 'last date' and program names, which keyword search handles very well. BM25 is fast, needs no extra model and is easy to explain. Retrieval only supplies evidence; the main answers come from structured data. Embeddings are a possible later upgrade.

Q10. How do you prevent hallucinations?

Four ways: the facts come from the rule engine, not the model; the system prompt forbids inventing numbers and requires citations; if the model call fails or refuses, a deterministic answer is shown; and questions outside the scope are redirected instead of answered.

Q11. What happens if there is no API key or the AI service is down?

Everything still works. The engine's result is formatted by a built-in renderer. The AI only improves the wording.

Q12. How does the assistant handle questions it shouldn't answer?

A decision layer labels them. Exceptions such as missed deadlines or appeals, and sensitive topics, are ESCALATE - the student gets the admission office contacts. Non-admission topics are OFF_TOPIC. Missing details trigger ASK_INFO.

C. Questions about agents

Q13. What is an AI agent? Is your project an agent?

An agent observes its environment, decides and acts towards a goal, usually in a loop, using tools. The update agent is one: every day it observes 23 websites, compares them with the last copy, decides what is new or broken, acts by saving the text and drafting updates, and reports through a pull request.

Q14. Why doesn't the agent update the data automatically?

Because fees, dates and minimum marks are official facts. A misread notice could mislead thousands of students. The agent finds and highlights the change; a person verifies it and approves. That is the 'human in the loop' principle.

Q15. What is MCP and why did you add it?

The Model Context Protocol is a standard for giving AI assistants tools. My MCP server exposes functions such as check_admission and get_test, so an assistant like Claude Desktop can answer admission questions using my verified engine rather than guessing.

D. Questions about engineering and testing

Q16. How did you test the system?

With 49 automated tests covering mark parsing, each eligibility rule and edge case, the merit aggregate, chat routing, the API and the update agent's detectors. I also ran sample notebooks, tested the portal in a browser, and simulated a changed web page to prove the agent detects new dates.

Q17. Give an example of a bug you found and fixed.

MDCAT 95 was first read as 95 percent instead of 95 out of 180, which would have marked a failing student as eligible for MBBS. A test caught it, and I changed the rule so that for tests with a fixed maximum a plain number means marks obtained. Another: 'must' is an English word, so 'which documents must I attach' was treated as MUST university; now only the uppercase acronym counts.

Q18. How is the project structured?

Data (raw sources and YAML rule files) is separate from logic (engine, router, decision layer) and from interfaces (portal, API, MCP). This separation means data can be updated without touching code, and each part can be tested on its own.

Q19. How do you handle a website that is down?

The JAC portal returns a block page. I do not bypass protections; I mark the site as not working, give students working official alternatives, never link to it, and the update agent reports when it comes back.

E. Questions about ethics and limits

Q20. What are the limitations of your system?

The chance label is a heuristic, because closing merits are not published. Some scanned fee sheets are not yet transcribed. The JAC and AJKMC websites are unavailable. And the data must be refreshed every admission cycle, which the agent helps with.

Q21. How do you deal with personal data?

The eligibility check needs only marks and group, nothing that identifies the student, and nothing is stored. When I found an official statistics file that listed candidates' names, I kept only the averages and deleted the file.

Q22. Could this give wrong advice? What protects students?

Every requirement is shown with its source, third-party facts are labelled, exceptions go to the admission office, and students are told to confirm final details with the institution. Human approval of data changes adds another safety layer.

F. General and behavioural questions

Q23. What did you learn from this project?

That reliable AI products are mostly about good data and clear rules. The model is a small part; knowing when not to use it, how to show evidence and how to keep information current matters more.

Q24. How would you explain your project to a non-technical person?

"It's like a knowledgeable counsellor who has read every university's admission rules. You tell it your marks, and it tells you where you can apply, what you still need and when to apply - and shows you the official page for each answer."

Q25. How did you manage your time and priorities?

I started with the most common student need - 'Where can I apply with my marks?' - then added tests, steps and dates, then reliability work (tests, the update agent). When the scope grew, I cut features that did not serve admissions.

Tips for answering well

- Use the **STAR** method for behavioural questions: Situation, Task, Action, Result.
- Always mention one concrete number: 40 programs, 23 sources, 49 tests, MDCAT 99/180.
- Be honest about limitations - interviewers value it.
- Prepare a short live demo: one eligibility check and one chat question.
- Practise the 60-second pitch aloud until it feels natural.

6. Glossary

Term	Meaning
HSSC / SSC	Higher Secondary School Certificate (intermediate, 12 years) / Secondary School Certificate (matric, 10 years).
MDCAT	Medical & Dental Colleges Admission Test by PM&DC. 180 MCQs, no negative marking. MBBS needs 99, BDS 90.
ECAT	Engineering College Admission Test by UET. 400 marks (100 MCQs × 4), negative marking. PEC minimum 132 (33%).
SEET	MUST's State Entry Examination Test for engineering and engineering technology.
LAT	HEC Law Admission Test, needed for LLB.
PM&DC	Pakistan Medical & Dental Council, the regulator for medical and dental admissions.
PEC	Pakistan Engineering Council, which accredits engineering programs.
JAC	Joint Admission Committee for MBBS/BDS in AJK public colleges (its website currently does not work).
SZABMU	Shaheed Zulfiqar Ali Bhutto Medical University, Islamabad; conducts MDCAT for AJK.
State Subject	AJK residency certificate required for AJK-national seats.
Merit aggregate	Weighted score for MBBS/BDS: 50% MDCAT + 40% HSSC + 10% SSC.
YAML	A simple text format for structured data, used for the rule book.
BM25	A keyword-based ranking method for searching documents.
RAG	Retrieval-augmented generation: search documents, then answer from what was found.
MCP	Model Context Protocol: a standard way to give AI assistants tools.
Pull request	A proposed change on GitHub that a person reviews before it is merged.